

# Page 1: What Is CRAFT Trying to Do?

Imagine a maze with little robots. Each robot starts somewhere and needs to walk to its own special star.

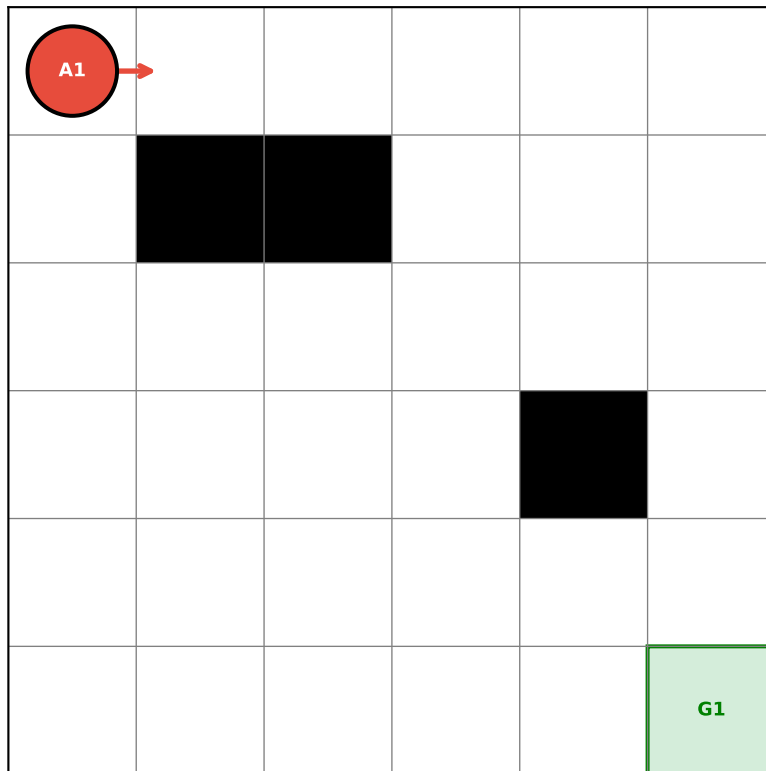
Easy if there's only ONE robot – just walk toward the star!

Hard when there are MANY robots – they bump into each other and get stuck.

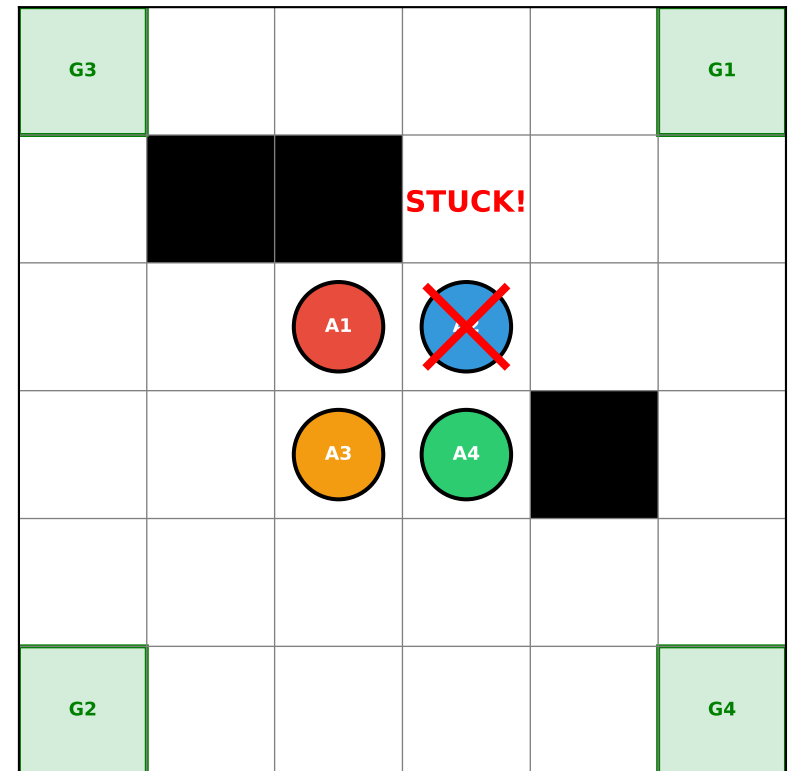
A smart brain (called the UNet) already knows how to guide ONE robot.  
But it was never taught to share the road with others.

CRAFT's job: teach it to cooperate – without rebuilding the whole brain.

**One robot — easy!**



**Four robots — crash!**



*CRAFT = add a tiny helper on top of the frozen brain that says:  
'Hey A2, careful — A1 is right there!'*

# Page 2: What the Brain Sees — 6 Layers of Information

Before a robot can decide where to move, it has to LOOK at the world.  
The world is given as 6 pictures stacked on top of each other – like 6 transparencies on an overhead projector, each showing something different.

## Ch 0 Walls

Black = wall  
White = free

'Where can  
I go?'

## Ch 1 Robot positions

Each robot gets  
a different number

'Who is where?'

## Ch 2 Goal positions

Each robot's star  
gets its number

'Where does  
each go?'

## Ch 3 Distance to star

Close = bright  
Far = dark

'How far am  
I from home?'

## Ch 4 Distance slope ↔

Bright = goal is  
to the right  
Dark = to the left

'Which column?'

## Ch 5 Distance slope ↕

Bright = goal is  
below  
Dark = above

'Which row?'

## Page 3: The Frozen Brain — UNet

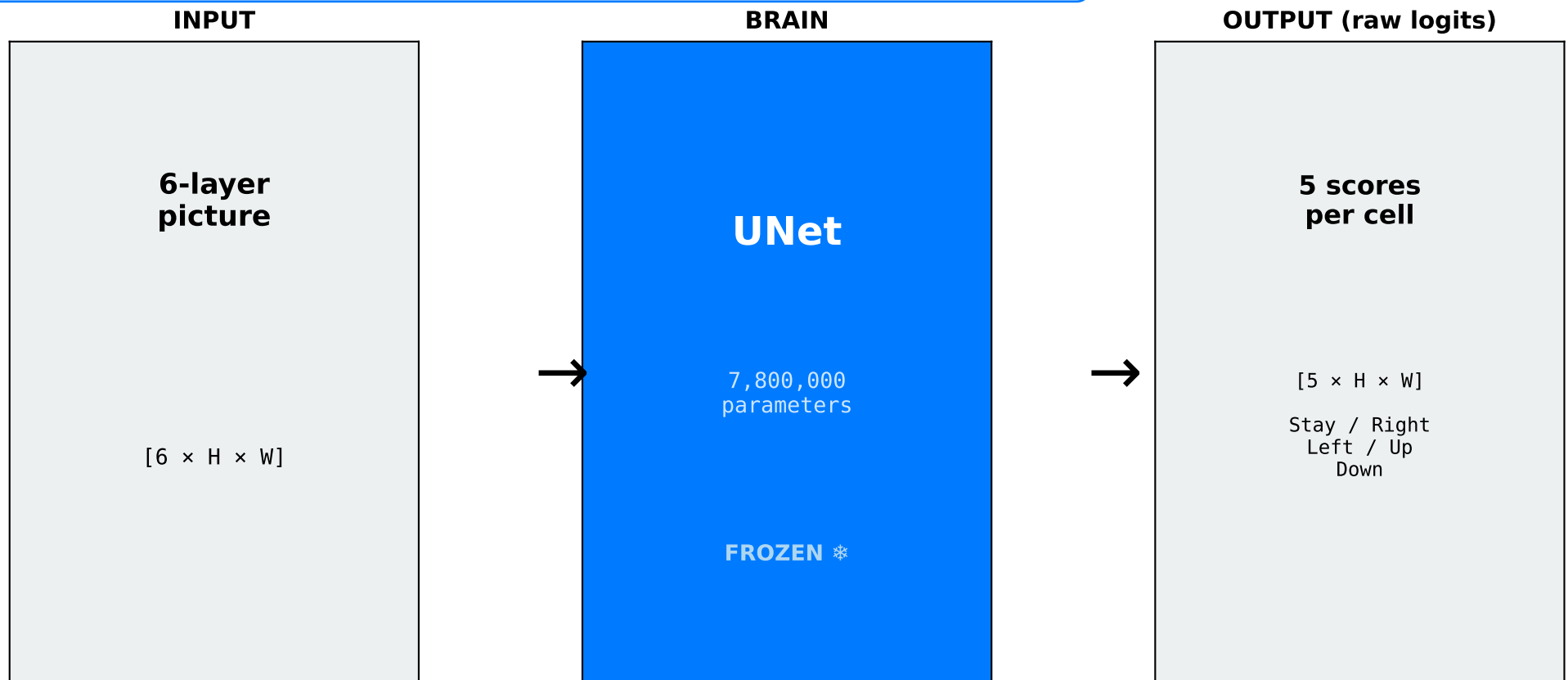
The UNet is a big neural network with 7.8 MILLION numbers inside it.  
Think of it as a very experienced navigator who has seen millions of mazes.

It reads the 6-layer picture and outputs ONE thing:  
For EVERY single cell in the grid, 5 scores:

[Stay, Go Right, Go Left, Go Up, Go Down]

Higher score = 'this direction is better from this cell.'

We do NOT change the UNet. It's frozen, like a textbook.  
CRAFT only adds a thin sticky note on top.



Problem: UNet was trained alone – it ignores other robots.  
The scores it gives don't say 'but watch out, A2 is going left!'

## Page 4: The Helper — Heuristic Phi (Traffic Sensor)

CRAFT adds a HELPER that computes a 'danger score' (called phi,  $\phi$ ) for every robot.  
High  $\phi$  = 'this robot is in a crowded or dangerous spot – others should avoid it.'  
Low  $\phi$  = 'this robot is fine, nothing special going on.'

The helper uses THREE hand-crafted signals (no learning needed – pure math):

### SIGNAL 1: Local Density

Count how many other robots are within 3 steps of this robot.

Many neighbors → high cost.  
Alone → low cost.

Weight: 2.0

Analogy: a crowded hallway vs. an empty one.

### SIGNAL 2: Bottleneck Score

Some cells on the map are CHOKES POINTS – almost every path passes through them.

Pre-computed once per map using graph betweenness.

Weight: 3.0

Analogy: a narrow bridge everyone must cross.

### SIGNAL 3: Direction Conflict

Is another robot walking straight toward THIS robot?

If yes → head-on collision risk → raise the cost!

Weight: 5.0

Analogy: two cars on a one-lane road facing each other.

$$\phi_i = 2.0 \times \text{density} + 3.0 \times \text{bottleneck} + 5.0 \times \text{conflict}$$

# Page 5: Cost Shaping — How the Helper Bends the Brain's Scores

The UNet gives raw scores. The helper gives a danger number  $\phi$  per robot.  
Cost shaping COMBINES them:

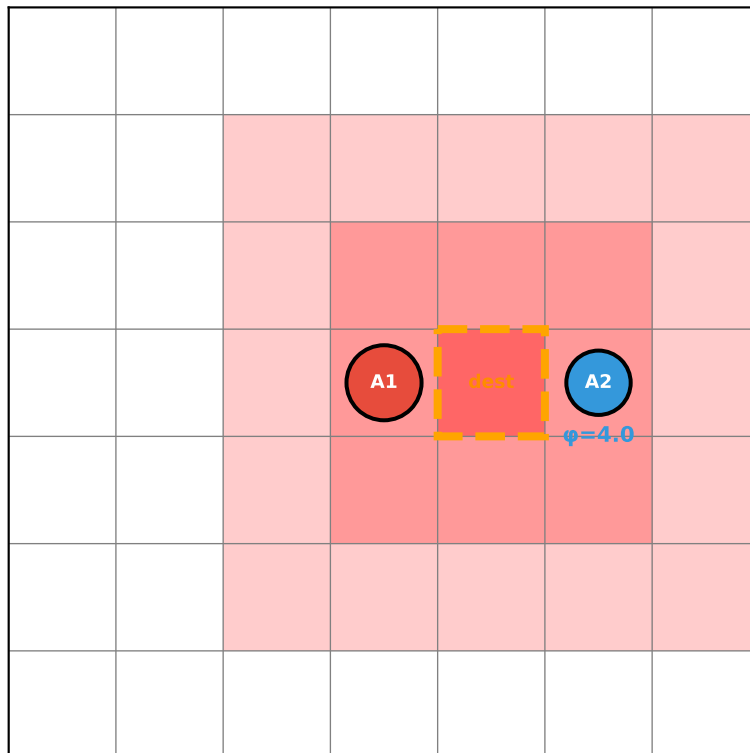
```
shaped_score[robot, action] = UNet_score - penalty
```

```
penalty = sum over nearby robots j of: weight(dist) ×  $\phi_j$   
weight(dist) = max(0, (radius+1 - dist) / (radius+1))
```

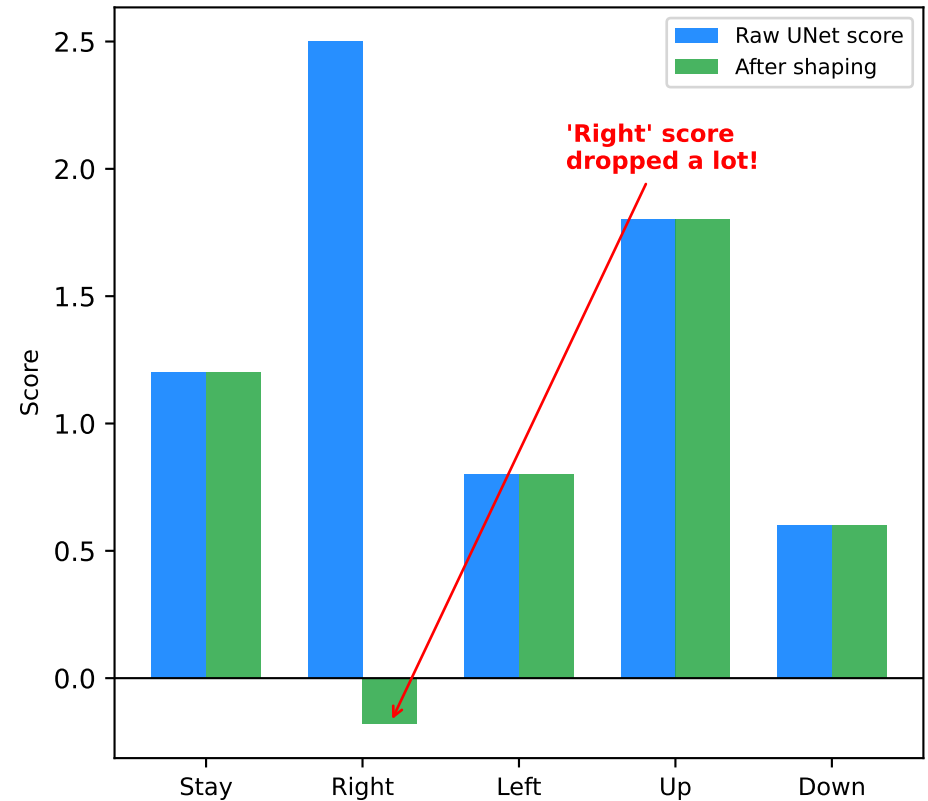
```
dist=0 → weight=1.0    dist=1 → weight=0.67    dist=2 → weight=0.33
```

If robot A1 is thinking about going RIGHT and A2 ( $\phi=4.0$ ) is 1 step to the right:  
penalty  $\approx 0.67 \times 4.0 = 2.68$  subtracted from the 'go right' score.  
The 'go up' score is untouched. So A1 will probably go up instead. ✓

**A1 thinking about going RIGHT**  
(destination = orange cell)



**Scores before & after shaping**  
(A2 with  $\phi=4.0$  is 1 step right of dest)



## Page 6: PPO — Learning From Experience

The heuristic  $\phi$  uses fixed math. But we can also LEARN better weights.  
PPO (Proximal Policy Optimization) is a technique for training by trial and error:

1. Let the robots run around the maze for a while (ROLLOUT)
2. Give rewards: +1 when a robot reaches its star, -small every step
3. Ask: which decisions led to good outcomes? (GAE – see below)
4. Nudge the policy a tiny bit toward those decisions (UPDATE)
5. Repeat thousands of times

We train TWO things with PPO:

- Policy (action scores) – the shaped UNet + fine-tuned top layers
- Value head – a tiny MLP that predicts 'how good is this situation?'

GAE – Generalized Advantage Estimation ( $\gamma=0.99$ ,  $\lambda=0.95$ )

'Did this action turn out better or worse than I expected?'

advantage = (actual reward I got) – (what the value head predicted)

$\gamma=0.99$ : care about future rewards almost as much as immediate ones

$\lambda=0.95$ : balance between short-horizon accuracy and long-horizon reach

Positive advantage → do this action MORE often

Negative advantage → do this action LESS often

PPO clip ( $\epsilon=0.2$ ): Never change the policy too much in one step.  
Like saying 'improve yourself, but don't completely reinvent yourself overnight.'

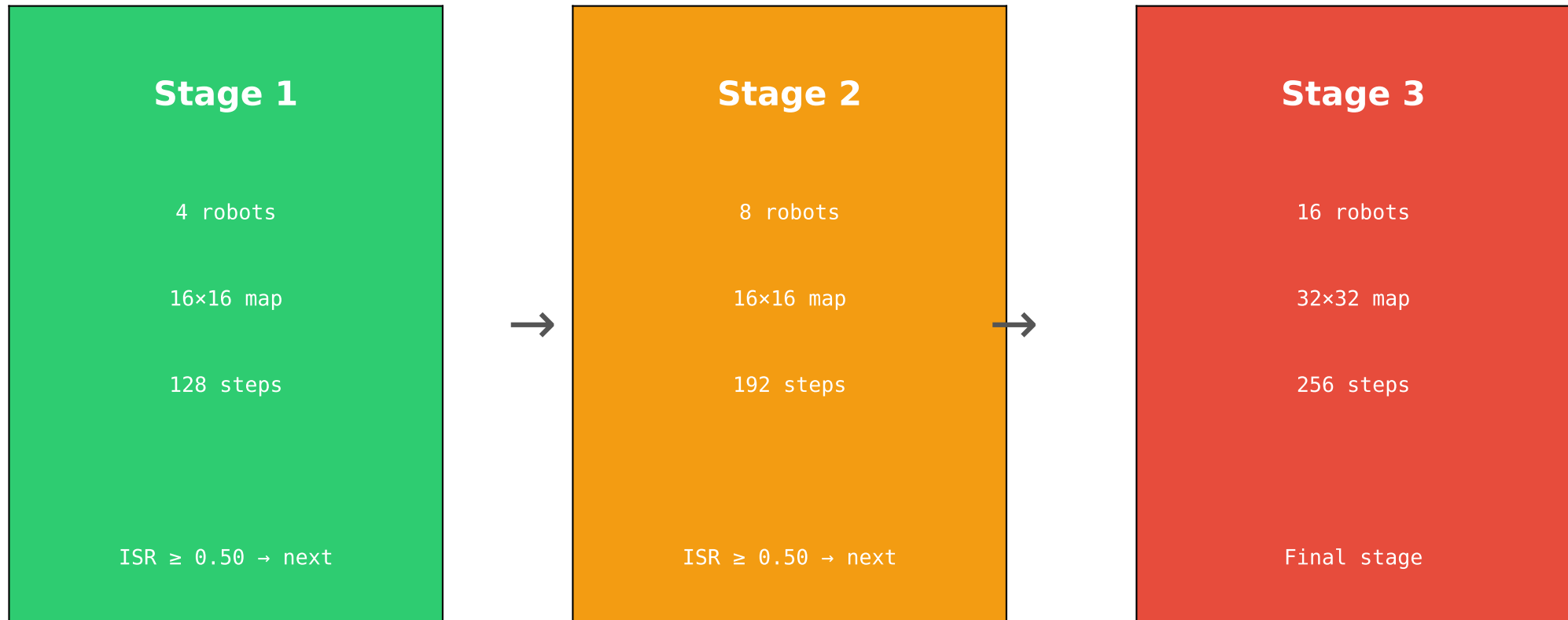
## Page 7: Curriculum Learning — Start Easy, Get Harder

We don't throw the robots into the hardest level immediately.  
Like a video game, we start EASY and get HARDER when they're ready.

We measure readiness with ISR — Individual Success Rate:

$$\text{ISR} = (\text{number of robots that reached their star}) / (\text{total robots})$$

When  $\text{ISR} \geq 0.50$  (50% success), we move to the next level.



Why curriculum?

If you start with 16 robots on a big map, the reward signal is so noisy that the model barely learns anything. Starting small makes learning stable.

## Page 8: Before Training — Diagnosing the Problem

Before fixing anything, we first PROVE that the problem actually exists. We run the frozen UNet on many maps and log every decision it makes.

Then we ask five questions:

Q1: Does the UNet disagree with the expert (LaCAM) MORE when there are more robots nearby?  
→ Measures: does density predict mistakes?

Q2: Where on the map do mistakes cluster?  
→ Spatial heatmap of errors.  
→ Do errors bunch near chokepoints?

Q3: Do cells with high 'betweenness' (bottlenecks) have more mistakes than open cells?  
→ Confirms bottleneck hypothesis.

Q4: Can density ALONE predict whether the UNet will make a mistake? (AUC of logistic regression)  
→ AUC > 0.6 confirms density matters.

Q5: Is there systematic directional conflict?  
Do two robots often 'want' the same cell at the same time?  
→ Measures head-on collision rate.

All five analyses are implemented in diagnostic.ipynb (Colab) and diagnostic\_local.ipynb (local). Results go into results/diagnostic\_report.tex.

## Page 9: The Full Pipeline at a Glance

— ENVIRONMENT (envs/pogema\_railgun\_env.py)  
Grid world. Produces the 6-layer picture every step.  
Runs up to 256 steps per episode.

— FROZEN UNet (pretrained RAILGUN checkpoint)  
Reads the 6-layer picture.  
Outputs 5 raw scores per cell. Never updated.

— HEURISTIC PHI (tools/heuristic\_cost.py)  
Computes  $\phi$  per robot from density + bottleneck + conflict.  
Subtracts proximity-weighted penalty from the raw UNet scores.

— ACTION SAMPLING (train\_rl.py)  
Each robot picks its action from the shaped scores.  
Stochastic during training, greedy during evaluation.

— PPO UPDATE (train\_rl.py – ppo\_update())  
Collect T-step rollout → compute GAE → update policy + value head.  
Gradient clipping at 0.5. Entropy bonus 0.01 keeps exploration alive.

— CURRICULUM CHECK (train\_rl.py – evaluate\_isr())  
Every eval\_interval: measure ISR. Save checkpoint if improved.  
Advance to next curriculum stage when  $\text{ISR} \geq \text{threshold}$ .

# Page 10: Key Numbers & Vocab Cheatsheet

## VOCAB

|                |                                                                          |
|----------------|--------------------------------------------------------------------------|
| UNet           | Big frozen brain (7.8M params). Reads map, gives direction scores.       |
| Phi ( $\phi$ ) | Danger number per robot. High = congested / bottleneck / conflict.       |
| Cost shaping   | Subtract a $\phi$ -based penalty from UNet scores before picking action. |
| PPO            | Trial-and-error training algorithm.                                      |
| GAE            | Method to figure out which decisions were actually good ones.            |
| ISR            | Individual Success Rate = how many robots reached their star.            |
| Curriculum     | Train easy first, get harder when ISR passes 50%.                        |

## KEY NUMBERS

|           |                                                                                       |
|-----------|---------------------------------------------------------------------------------------|
| 7,800,000 | Parameters in the frozen UNet backbone                                                |
| 6         | Input channels per cell (walls, agents, goals, dist, grad_x, grad_y)                  |
| 5         | Actions per robot (stay, right, left, up, down)                                       |
| 3         | Phi signals (density $\times 2.0$ , bottleneck $\times 3.0$ , conflict $\times 5.0$ ) |
| 2         | Proximity radius for cost shaping (L-inf distance)                                    |
| 3         | Curriculum stages (4 $\rightarrow$ 8 $\rightarrow$ 16 robots)                         |
| 0.50      | ISR threshold to advance curriculum stage                                             |
| 0.99      | $\gamma$ – how much we care about future rewards                                      |
| 0.95      | $\lambda$ – GAE balance between short/long horizon                                    |
| 0.20      | PPO clip $\epsilon$ – maximum policy change per step                                  |
| 3e-5      | Learning rate                                                                         |

To run training: `python train_rl.py --config configs/rl_ppo.yaml`  
To generate PDFs: `python docs/craft_explained.py`